

SOP 10 — Ingest News

Camada A.N.I.: Instruments-Tools (ingest_news.py) · Versão: 1.0 (Protocol 0, 2026-09-09)

Golden Rule: mudou a lógica → atualiza o SOP → só então o código.

Objetivo

Coletar notícias novas **somente das fontes da allowlist** (ver `gemini.md`), transformar cada item no contrato `RawInput.NewsItem` e gravar o lote em `.tmp/` para a etapa de classificação. Nunca republica texto integral e nunca scrapar atrás de paywall.

Quando roda

- Cron daily ingest 06:30 (lote principal do dia).
- Cron refresh 12:00 e 18:00 (apenas itens novos desde a última coleta).
- Manualmente, na Phase L, via `tools/ping_rss.py` (1 item por fonte).

Input

- Allowlist de fontes de `gemini.md` (seção “Allowlist inicial de fontes (news)”).
- Para cada fonte: feed RSS/HTTP disponível (validado na Phase L) **ou** decisão humana de rota alternativa (ex.: RSSHub self-host — nunca automático).
- Registro do que já foi coletado: `.tmp/seen/<data>.json` (bancada) e aba cloud `news_cards / sources` quando disponível.

Output

- `RawInput.NewsItem[]` → `.tmp/news/<YYYY-MM-DD>.json` (lote com `fetch_at` no momento da coleta).
- Cada item: `source_id`, `source_name`, `url`, `title_raw`, `summary_raw`, `published_at`, `fetch_at` (ISO com offset -03:00).

Passos determinísticos

1. Ler allowlist de fontes. **Ignorar qualquer domínio fora da lista** (regra do blueprint; logar o evento em `progress.md`).
2. Para cada fonte da lista: buscar o feed via HTTP GET com timeout de 15s.
3. Respeitar `robots.txt` e politeness de 1 request a cada 5s por domínio.
4. Parsear o feed (RSS/Atom/JSON Feed ou HTML conforme a fonte — o formato de cada fonte é fixado na Phase L e registrado aqui como apêndice).
5. Normaliz-
ar: codificar como UTF-8 (fontes BR podem entregar ISO-8859-1); descartar HTML; montar `published_at` em America/Sao_Paulo com offset explícito.
6. Filtrar: manter só itens com `title_raw` e `url` não vazios e com `published_at` dentro da janela da coleta (últimas ~24h na ingest 06:30).

7. Dedupe técnico de coleta: remover URLs já vistas (`.tmp/seen/` + aba `news_cards` na cloud); se a cloud não estiver acessível, registrar pendência — nunca tratar `.tmp/` como verdade definitiva.
8. Gravar lote em `.tmp/news/<YYYY-MM-DD>.json` e atualizar `.tmp/seen/`.
9. Reportar JSON de saída: `{ "source": "...", "fetched": n, "new_items": n, "errors": [...] }.`

Edge cases

- Feed fora do ar / timeout → 3 tentativas com backoff (5s, 15s, 30s); persistindo falha, registrar em `progress.md` + `findings.md` (fonte marcada como falha do dia — entra no Maintenance Log da fase T). As outras fontes seguem.
- Item sem `title_raw` ou `url` → descartar e logar.
- Domínio fora da allowlist aparecer no feed → ignorar item e logar aviso.
- Fonte sem RSS: **não inventar rota**; registrar em `findings.md` e pedir decisão humana (candidato: RSSHub self-host).
- Item atrás de paywall → não coletar (proibido no blueprint).
- Dois itens iguais vindos da mesma fonte → manter o mais recente; dedupe semântico entre fontes é do SOP 20.

Rate limits / limites conhecidos

- Politesse: 1 req/5s por domínio; timeout 15s; retries máx. 3.
- Nenhum limite oficial conhecido das fontes da allowlist (validar na Phase L e registrar aqui).

Testes

- Fixture: `fixtures/news_bundle.json` — a tool deve aceitar o bundle como entrada sem tocar em rede (modo dry-run com `--fixture`).
- Phase L: `tools/ping_rss.py` — 1 item real de cada fonte allowlist; registrar o resultado em `progress.md`.